

Manual de Usuario

Analizador Sintáctico LL (Wison)

Sistema de Análisis Sintáctico Descendente

Tabla de Contenidos

1. [Introducción](#)
2. [¿Qué es un Analizador Sintáctico LL?](#)
3. [Primeros Pasos](#)
4. [Interfaz de Usuario](#)
 - [4.1 Panel de Configuración](#)
 - [4.2 Panel de Analizadores](#)
 - [4.3 Panel de Pruebas](#)
 - [4.4 Panel de Resultados](#)
5. [Lenguaje Wison](#)
 - [5.1 Estructura General](#)
 - [5.2 Bloque Léxico \(Lex\)](#)
 - [5.3 Bloque Sintáctico \(Syntax\)](#)
 - [5.4 Comentarios](#)
6. [Creación de un Analizador](#)
7. [Árboles de Derivación](#)
8. [Manejo de Errores](#)

1. Introducción

Bienvenido al **Analizador Sintáctico LL (Wilson)**, una herramienta diseñada para la construcción y evaluación de analizadores sintácticos descendentes. Esta aplicación permite definir gramáticas libres de contexto y validar cadenas de entrada de forma visual e interactiva.

Características Principales

- **Interfaz Web Intuitiva:** Diseño moderno y fácil de usar
 - **Análisis en Tiempo Real:** Validación inmediata de gramáticas
 - **Visualización de Árboles:** Representación gráfica de derivaciones
 - **Editor Integrado:** Sintaxis resaltada
 - **Detección de Errores:** Reportes detallados de errores léxicos, sintácticos y semánticos
-

2. ¿Qué es un Analizador Sintáctico LL?

Un **analizador sintáctico LL** es un tipo de analizador descendente que procesa las entradas de **izquierda a derecha** y construye una derivación por la **izquierda**.

Requisitos de una Gramática LL

Para que una gramática pueda ser reconocida por un analizador LL, debe cumplir:

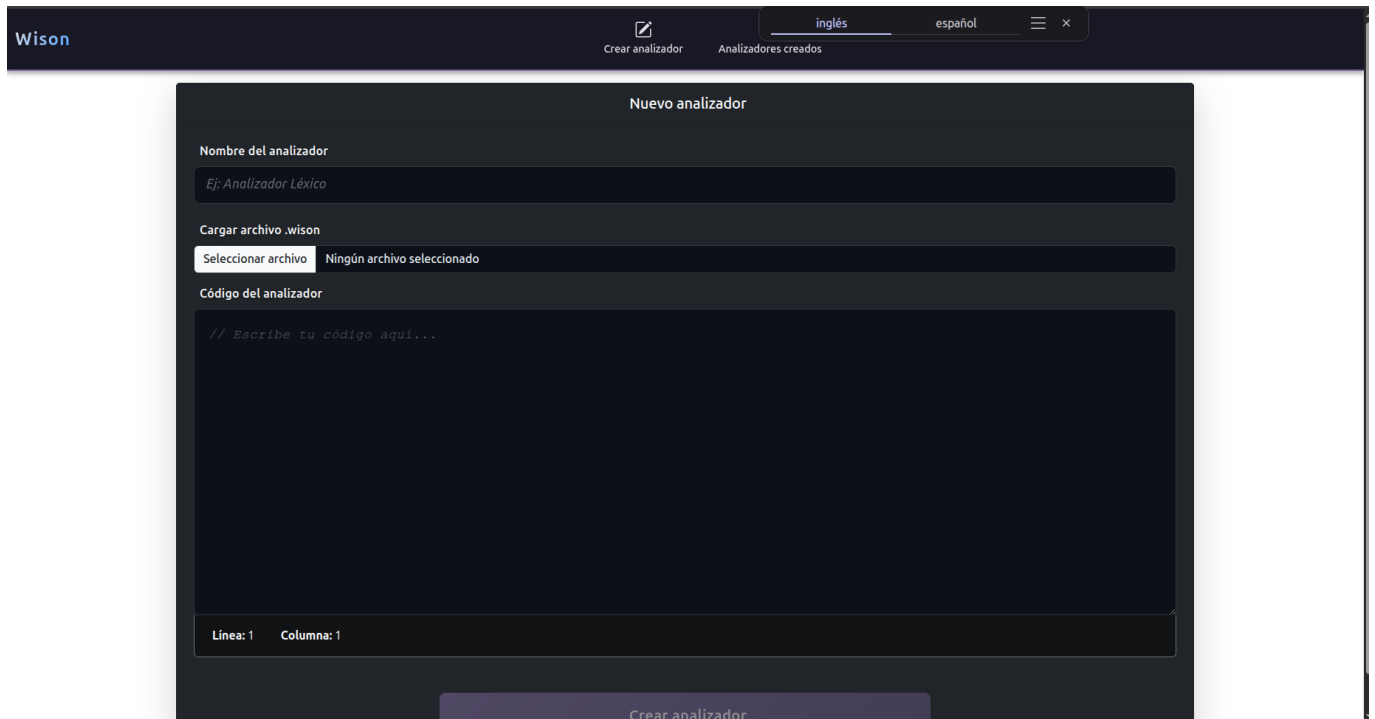
Requisito	Descripción
Sin Ambigüedad	Cada cadena debe tener una única derivación
Sin Recursividad Izquierda	No debe haber producciones de la forma $A \rightarrow A \alpha$
Factorizada	Debe aplicarse factorización cuando sea necesario

⚠ **Importante:** Si una gramática no cumple estos requisitos, la tabla de análisis sintáctico tendrá colisiones y no podrá ser procesada.

3. Primeros Pasos

Acceso a la Aplicación

1. Abra su navegador web preferido
2. Ingrese a la URL de la aplicación, `http://localhost:4200/wison`
3. La interfaz principal se cargará automáticamente



4. Interfaz de Usuario

La interfaz está organizada en cuatro áreas principales que facilitan el trabajo con analizadores sintácticos.

4.1 Panel de Nuevo analizador

Este panel permite escribir la definición de un nuevo analizador sintáctico y léxico.

Funcionalidades:

- **Editor de Código:** Área de texto para entrada de un nuevo analizador
- **Cargar Archivo:** Botón para importar archivos `.wison`
- **Crear Analizador:** Botón para guardar el analizador validado

Nuevo analizador

Nombre del analizador
nuevo analizador ✓

Cargar archivo .wison
Seleccionar archivo GRAMÁTICA COMPLEJA PARA.wison

Código del analizador

```
No_Terminal %_Block;
No_Terminal %_Declarations;
No_Terminal %_Declaration;
No_Terminal %_Declaration_Prime;

# Declaraciones de constantes
No_Terminal %_Const_Decl;
No_Terminal %_Const_List;
No_Terminal %_Const_List_Prime;
No_Terminal %_Const_Item;
```

✓
Linea: 515 Columna: 1

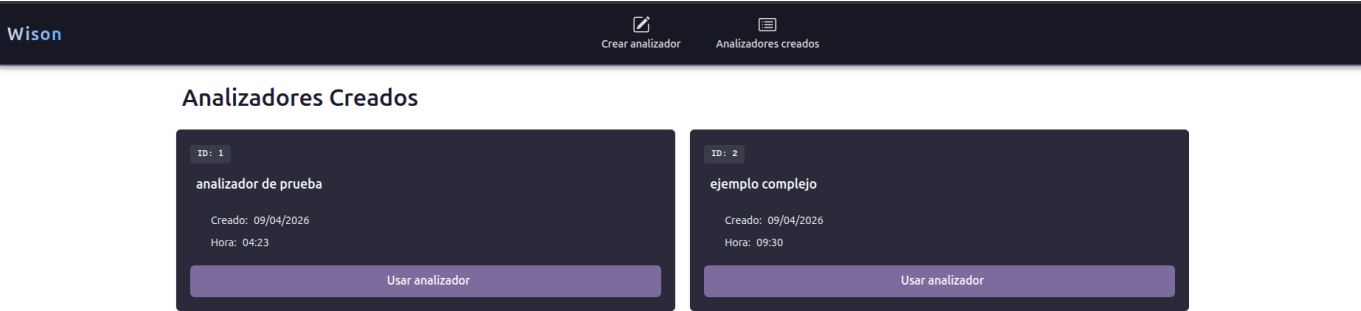
Crear analizador

Opciones Disponibles

Botón	Función
Nuevo	Limpia el editor para crear una nueva configuración
Cargar	Abre un selector de archivos para importar código Wison
Crear	Genera y guarda el analizador sintáctico

4.2 Panel de Analizadores

Muestra todos los analizadores sintácticos que han sido creados y están disponibles para usar.

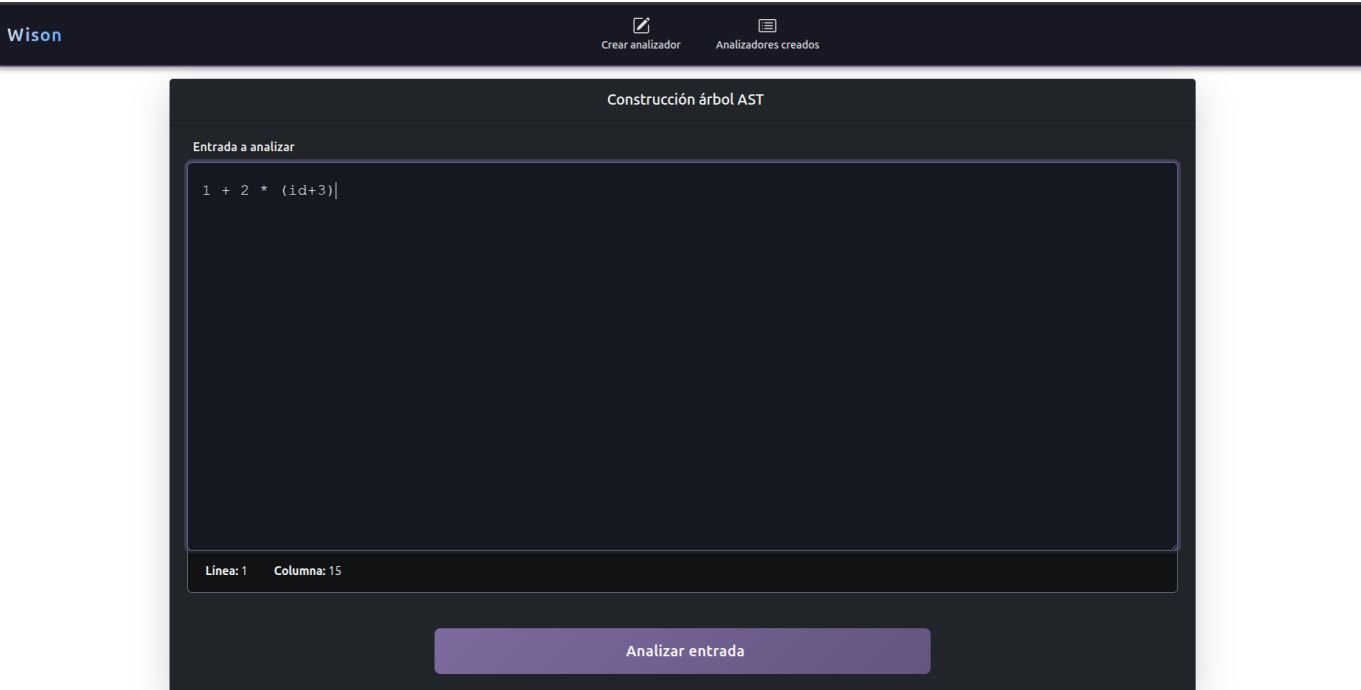


Información mostrada:

- Nombre del analizador
- Fecha de creación
- Usar analizador

4.3 Panel de Pruebas

Permite ingresar cadenas de entrada para ser analizadas por el analizador seleccionado.

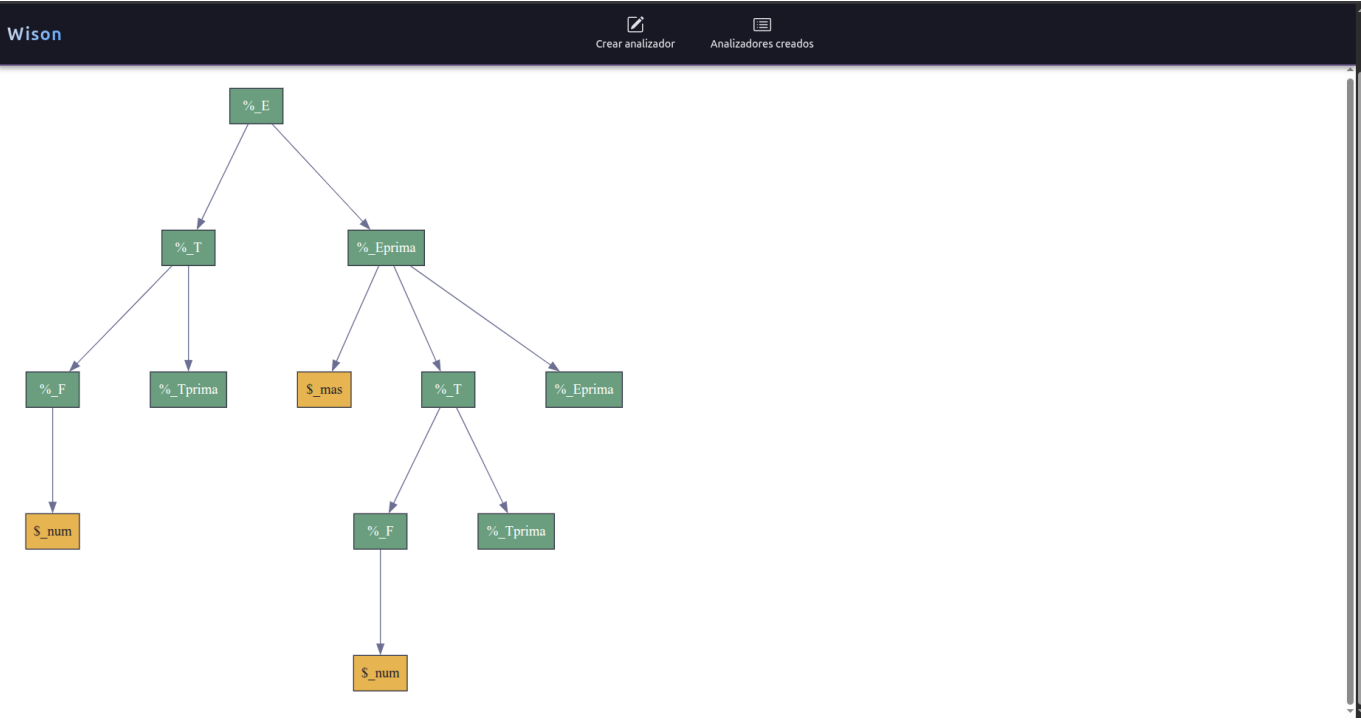


Elementos:

- **Campo de Entrada:** Donde se escribe la cadena a validar
- **Botón Analizar:** Inicia el proceso de análisis

4.4 Resultados

Visualiza el el árbol de derivación o los errores encontrados.



The screenshot shows the Wison interface with a dark theme. A dialog box titled 'Construcción árbol AST' is open. It contains an input field with the text '1 + 6 6'. Below the input field, there is a table titled 'Errores encontrados' with the following data:

TIPO	LÍNEA	COLUMNA	LEXEMA	DESCRIPCIÓN
Léxico	1	5	6	Caracter no reconocido.
Léxico	1	6	6	Caracter no reconocido.
Sintáctico	1	7	\$_EOF	Se esperaba: \$_id, \$_num, \$_para,

At the bottom of the dialog box, there is a button labeled 'Analizar entrada'.

Contenido:

- **Árbol de Derivación:** Representación gráfica de la derivación
- **Mensajes de Error:** Si la cadena no es aceptada
- **Tabla de Errores:** Detalle de errores encontrados

5. Lenguaje Wison

Wison es el lenguaje de configuración utilizado para definir analizadores sintácticos. Permite especificar tanto la parte léxica como la sintáctica de una gramática.

5.1 Estructura General

Todo archivo Wison debe seguir esta estructura básica:

```
# Comentario inicial

Wison ¿

Lex { :
    # Definiciones léxicas
: }

Syntax { { :
    # Definiciones sintácticas
: } }

?Wison

# Fin del archivo
```

 **Nota:** Las palabras reservadas `Wison`, `Lex`, `Syntax`, etc., son **case-sensitive** .

Delimitadores:

Elemento	Delimitador Inicio	Delimitador Fin
Estructura completa	<code>Wison ¿</code>	<code>?Wison</code>
Bloque léxico	<code>Lex { :</code>	<code>: }</code>
Bloque sintáctico	<code>Syntax { { :</code>	<code>: } }</code>

5.2 Bloque Léxico (Lex)

Define los **tokens** o **terminales** de la gramática mediante expresiones regulares.

Sintaxis de Declaración

```
Terminal $_NOMBRE <- EXPRESIÓN ;
```

Elementos Permitidos

1. Caracteres y Palabras Reservadas

Deben ir entre comillas simples o comillas normales:

```
Terminal $_Mas      <- '+' ;
Terminal $_Menos    <- '-' ;
Terminal $_FIN      <- 'FIN' ;
Terminal $_IF       <- 'if' ;
```

2. Secuencias de Caracteres

Solo se permiten estas dos formas especiales:

```
Terminal $_Letra    <- [aA-zZ] ;      # Alfabeto completo
Terminal $_Digito   <- [0-9] ;        # Dígitos del 0 al 9
```

3. Operadores Unarios

Operador	Significado	Ejemplo
*	Estrella de Kleene (0 o más)	<code>[0-9]*</code>
+	Cerradura positiva (1 o más)	<code>[0-9] +</code>
?	Opcional (0 o 1 vez)	<code>[0-9] ?</code>

4. Concatenación

Se utiliza con paréntesis:

```
Terminal $_Decimal <- ([0-9]*) ('.') ([0-9]+) ;
```

Ejemplo Completo de Bloque Lex

```
Lex { :
  /**
   * Definición de tokens para una calculadora simple
   */

  # Operadores
  Terminal $_Mas      <- '+' ;
  Terminal $_Menos    <- '-' ;
  Terminal $_Por      <- '*' ;
  Terminal $_Div      <- '/' ;

  # Paréntesis
  Terminal $_P_Ab     <- '(' ;
  Terminal $_P_Ce     <- ')' ;

  # Números
  Terminal $_Digito    <- [0-9] ;
  Terminal $_Numero    <- [0-9]+ ;
  Terminal $_Decimal   <- ([0-9]*) ('.') ([0-9]+) ;

  # Identificadores
  Terminal $_Letra     <- [aA-zZ] ;
  Terminal $_ID        <- [aA-zZ] ([aA-zZ] | [0-9]) * ;
: }
```

5.3 Bloque Sintáctico (Syntax)

Define las **reglas de producción** de la gramática.

Declaración de No Terminales

```
No_Terminal %_NOMBRE ;
```

⚠ **Importante:** Todos los no terminales deben declararse antes de usarse en las producciones.

Declaración del Símbolo Inicial

```
Initial_Sim %_NOMBRE ;
```

⚠ **Obligatorio:** Debe haber un símbolo inicial declarado.

Definición de Producciones

```
%_NoTerminal <= CUERPO_PRODUCCIÓN ;
```

Operador de Alternativa:

Se utiliza | para indicar múltiples opciones:

```
%_Expresion <= %_Termino | %_Expresion $_Mas %_Termino ;
```

Ejemplo Completo de Bloque Syntax

```
Syntax {{:  
    No_Terminal %_S ;  
    No_Terminal %_Expresion ;  
    No_Terminal %_Expresion_P ;  
    No_Terminal %_Termino ;  
    No_Terminal %_Termino_P ;  
    No_Terminal %_Factor ;  
  
    # Símbolo inicial  
    Initial_Sim %_S ;  
  
    %_S <= %_Expresion $_FIN ;  
  
    %_Expresion <= %_Termino %_Expresion_P ;  
  
    Expresion' | - Termino Expresion' | ε  
    %_Expresion_P <= $_Mas %_Termino %_Expresion_P  
                    | $_Menos %_Termino %_Expresion_P  
                    | ;  
  
    %_Termino <= %_Factor %_Termino_P ;  
  
    %_Termino_P <= $_Por %_Factor %_Termino_P  
                    | $_Div %_Factor %_Termino_P  
                    | ;  
  
    %_Factor <= $_P_Ab %_Expresion $_P_Ce  
                | $_Numero ;  
:}}
```

5.4 Comentarios

Wison soporta dos tipos de comentarios:

Comentarios de Línea

Todo lo que sigue a `#` en la misma línea es un comentario:

```
# Este es un comentario de línea
Terminal $_Mas <- '+' ; # Comentario al final de línea
```

Comentarios de Bloque

Para comentarios multilínea se usa `/** */`:

```
/**
 * Este es un comentario de bloque
 * Puede ocupar múltiples líneas
 * Útil para documentación extensa
 */
```

Convenciones de Nombres

Tipo	Prefijo	Ejemplo
Terminal	<code>\$_</code>	<code>\$_Numero, \$_Mas</code>
No Terminal	<code>%_</code>	<code>%_Expresion, %_S</code>

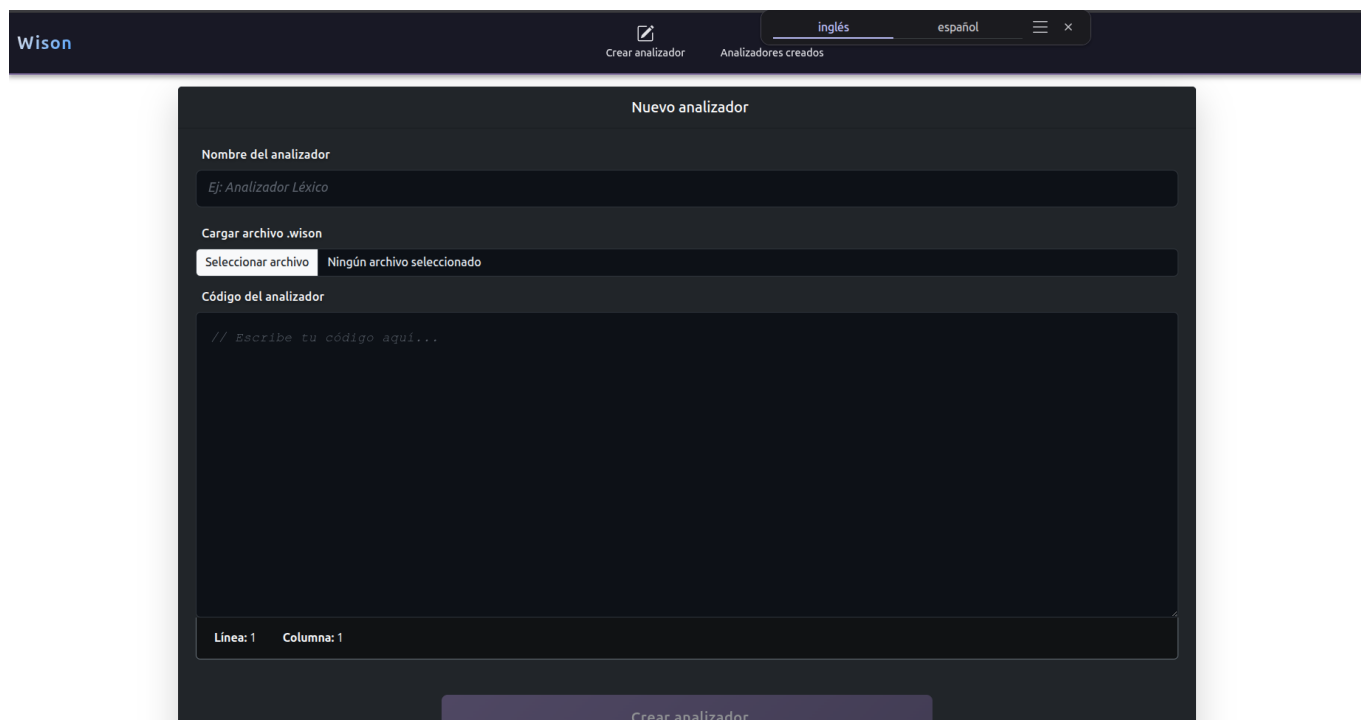
 **Regla:** Estos prefijos son obligatorios y permiten distinguir entre terminales y no terminales.

6. Creación de un Analizador

Siga estos pasos para crear un nuevo analizador sintáctico:

Paso 1: Escribir la Configuración

En el **Panel de Configuración**, escriba o pegue su código Wison junto con el nombre del analizador:



The screenshot shows the Wison web application interface. At the top, there's a dark header with the 'Wison' logo on the left, a 'Crear analizador' button with a pencil icon in the center, and a language selector on the right with 'inglés' and 'español' options. Below the header, the main content area is titled 'Nuevo analizador'. It contains three sections: 1. 'Nombre del analizador' with a text input field containing 'Ej: Analizador Léxico'. 2. 'Cargar archivo .wison' with a button 'Seleccionar archivo' and the text 'Ningún archivo seleccionado'. 3. 'Código del analizador' with a large text area containing the comment '// Escribe tu código aquí...'. At the bottom left of the text area, it shows 'Línea: 1' and 'Columna: 1'. At the bottom right, there is a 'Crear analizador' button.

```
Wison ¿

Lex {:
  Terminal $_Una_A <- 'a' ;
  Terminal $_Mas   <- '+' ;
  Terminal $_FIN   <- 'FIN' ;
:}

Syntax {{:
  No_Terminal %_S ;
  No_Terminal %_Suma ;

  Initial_Sim %_S ;

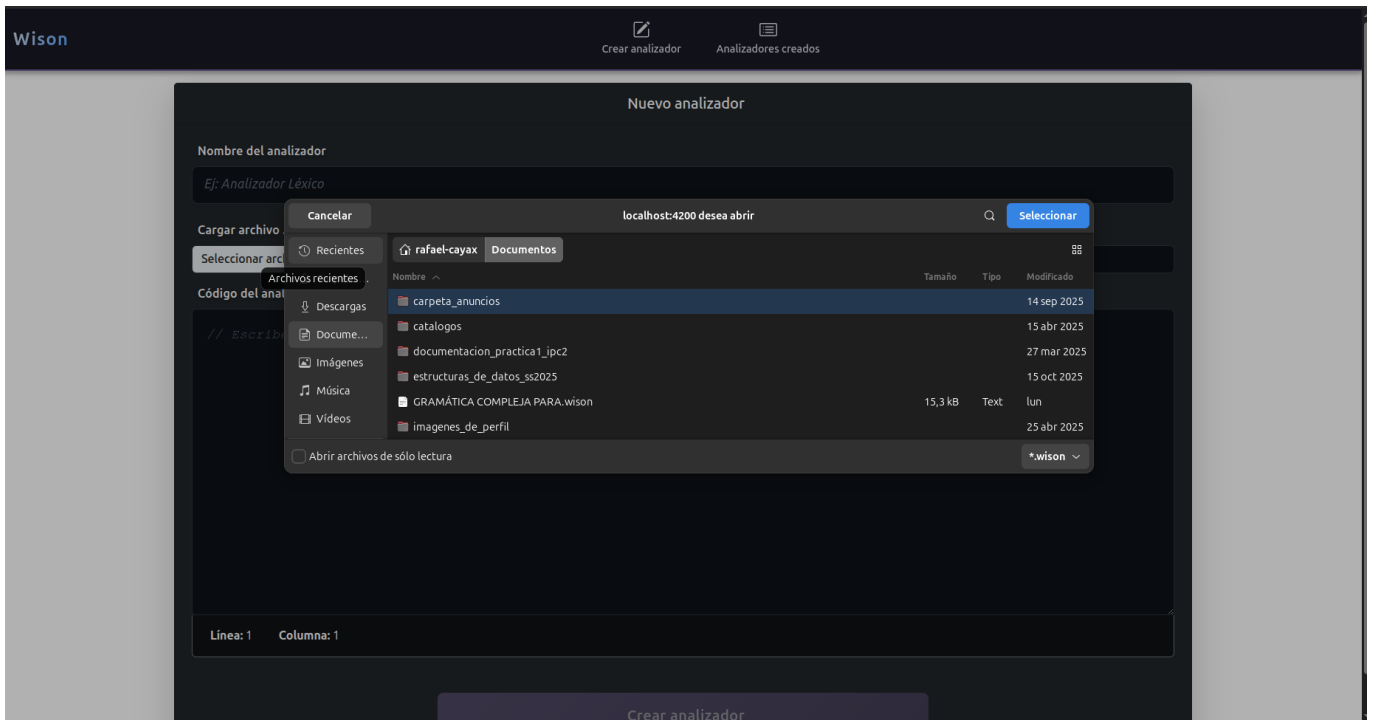
  %_S <= %_Suma $_FIN ;
  %_Suma <= %_Suma $_Mas $_Una_A | $_Una_A ;
:}}

?Wison
```

Paso 2: Cargar desde Archivo (Opcional)

Si prefiere cargar una configuración existente:

1. Haga clic en el botón **"Seleccionar Archivo"**
2. Seleccione un archivo **.wison** de su computadora
3. El contenido se cargará automáticamente en el editor



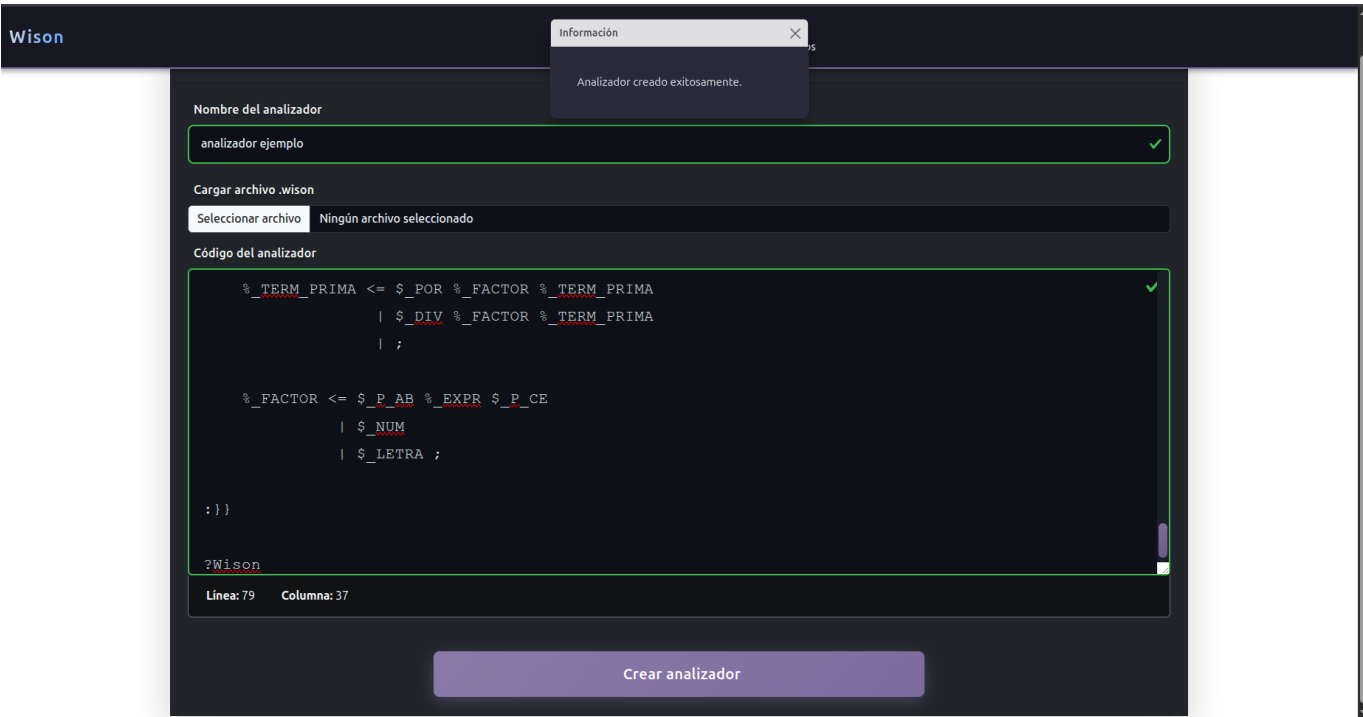
Paso 3: Evaluar la Configuración

Haga clic en el botón **"Crear analizador"** para validar:

- Sintaxis correcta del código Wison
- Ausencia de recursividad izquierda
- Gramática sin ambigüedades
- Factorización adecuada

Paso 4: Revisar Resultados

Si la evaluación es exitosa:



Si hay errores, se mostrará una tabla detallada:

Nombre del

analizador e

Cargar archi

Seleccionar

Código del a

%

:

}}}

?Wison

#

=====

FIN

Línea: 515

Errores encontrados

TIPO	LÍNEA	COLUMNA	LEXEMA	DESCRIPCIÓN
Semantico	190	16	%_Procedure_Call	Conflicto sobre el terminal \$_IDENTIFIER entre: %_Simple_Stmt <= %_Procedure_Call; %_Procedure_Call <= \$_IDENTIFIER %_Arg_List;
Semantico	232	16	%_Function_Call	Conflicto sobre el terminal \$_IDENTIFIER entre: %_Primary <= %_Function_Call; %_Function_Call <= \$_IDENTIFIER \$ _LPAREN %_Arg_List \$ _RPAREN;
Semantico	311	23	%_Subprog_Decls	Conflicto sobre el terminal \$_FUNCTION entre: %_Subprog_Decls <= %_Subprog_Decl %_Subprog_Decls; %_Subprog_Decls <= ;
Semantico	311	23	%_Subprog_Decls	Conflicto sobre el terminal \$_PROCEDURE entre: %_Subprog_Decls <= %_Subprog_Decl %_Subprog_Decls; %_Subprog_Decls <= ;
Semantico	383	20	%_Elsif_List	Conflicto sobre el terminal \$_ELSFIF entre: %_Elsif_List <= %_Elsif_Part %_Elsif_List; %_Elsif_List <= ;
				Conflicto sobre el terminal \$_ELSE entre:

Cerrar

Crear analizador

7. Árboles de Derivación

Los árboles de derivación muestran visualmente cómo una cadena es derivada desde el símbolo inicial.

Componentes del Árbol

- **Nodos Azules:** Símbolos no terminales
- **Nodos Naranjas:** Símbolos terminales
- **Flechas:** Relaciones de derivación

Ejemplo Visual

Para la gramática:

```
Wilson ¿

Lex { :
    Terminal $_id <- [aA-zZ]+;
    Terminal $_num <- [0-9]+;
    Terminal $_mas <- '+';
    Terminal $_por <- '*';
    Terminal $_parA <- '(';
    Terminal $_parC <- ')';
:}

Syntax {{ :

    No_Terminal %_E;
    No_Terminal %_Eprima;
    No_Terminal %_T;
    No_Terminal %_Tprima;
    No_Terminal %_F;

    Initial_Sim %_E;

    %_E <= %_T %_Eprima;

    %_Eprima <= $_mas %_T %_Eprima
                | ;

    %_T <= %_F %_Tprima;

    %_Tprima <= $_por %_F %_Tprima
                | ;

    %_F <= $_id
            | $_num
            | $_parA %_E $_parC;

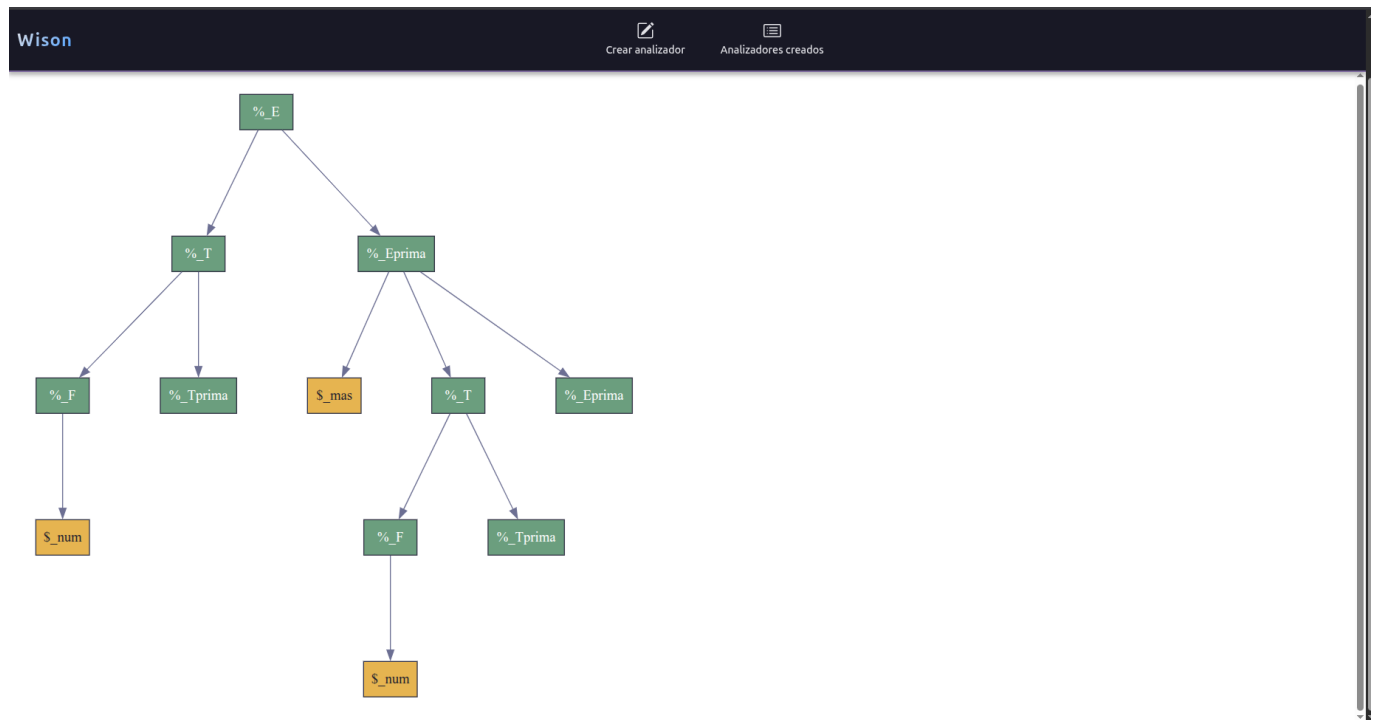
:}}}
```

? Wison

la entrada:

1+3

Árbol de derivación:



8. Manejo de Errores

El sistema detecta y reporta tres tipos de errores:

8.1 Errores Léxicos

Ocurren cuando se encuentran caracteres o secuencias no válidas.

Ejemplos:

Línea	Columna	Lexema	Tipo	Descripción
5	25	@	Léxico	Carácter no reconocido en la expresión regular

8.2 Errores Sintácticos

Ocurren cuando la estructura del código Wison es incorrecta.

Ejemplos:

Línea	Columna	Lexema	Tipo	Descripción
12	5	<=	Sintáctico	Falta declaración de no terminal antes de la producción
15	30	;	Sintáctico	Falta punto y coma al final de la declaración

8.3 Errores Semánticos

Ocurren cuando hay problemas lógicos en la definición de la gramática.

Ejemplos:

Línea	Columna	Lexema	Tipo	Descripción
18	10	%_Expr	Semántico	No terminal no declarado
22	15	\$_Desconocido	Semántico	Terminal no definido en el bloque Lex
25	5	%_S	Semántico	La gramática contiene recursividad izquierda
30	10	%_Factor	Semántico	No se ha declarado un símbolo inicial

Formato de la Tabla de Errores

Todos los errores se presentan en una tabla estructurada:

Estructura:

- **Línea:** Número de línea donde ocurre el error
- **Columna:** Posición del carácter en la línea
- **Lexema:** Token o símbolo que causó el error
- **Tipo:** Clasificación del error (Léxico, Sintáctico, Semántico)
- **Descripción:** Explicación detallada del problema